List of Experiments for DSA Lab Practical Exam- Winter 2025

1. Write a program to perform following operations on Arrays Data Structures: insertion, deletion, search, update, and display.

```
#include <stdio.h>

#define MAX 100

int main()
{
   int arr[MAX];
   int n = 0, choice, pos, value, i, key, found;

   while (1)
   {
      printf("\n--- ARRAY OPERATIONS MENU ---\n");
printf("1. Insert\n");        printf("2. Delete\n");
printf("3. Search\n");        printf("4. Update\n");
printf("5. Display\n");        printf("6. Exit\n");
printf("Enter your choice: ");
      scanf("%d", &choice);

      switch (choice)
      {
      case 1:   // Insert
         if (n == MAX)
         {
            printf("Array is full!\n");
            break;
         }
         printf("Enter position (0 to %d): ", n);
scanf("%d", &pos);            printf("Enter
value: ");
         scanf("%d", &value);

         for (i = n; i > pos; i--)
            arr[i] = arr[i - 1];

         arr[pos] = value;
         n++;
         printf("Element inserted successfully.\n");
break;

      case 2:   // Delete
```

```
        if (n == 0)
        {
           printf("Array is empty!\n");
           break;
        }
        printf("Enter position to delete (0 to %d): ", n - 1);
scanf("%d", &pos);

        for (i = pos; i < n - 1; i++)
           arr[i] = arr[i + 1];
         n--
;
        printf("Element deleted successfully.\n");
break;

      case 3:   // Search
found = 0;
        printf("Enter element to search: ");
scanf("%d", &key);

        for (i = 0; i < n; i++)
        {
           if (arr[i] == key)
           {
              printf("Element found at position %d\n", i);
found = 1;                  break;
           }
        }
        if (!found)
           printf("Element not found.\n");
        break;

      case 4:   // Update
        printf("Enter position to update (0 to %d): ", n - 1);
scanf("%d", &pos);          printf("Enter new value: ");
        scanf("%d", &value);

        arr[pos] = value;
        printf("Element updated successfully.\n");
        break;

      case 5:   // Display
        if (n == 0)
        {
```

```
            printf("Array is empty!\n");
         }
         else
         {
            printf("Array elements:\n");
            for (i = 0; i < n; i++)
printf("%d ", arr[i]);
printf("\n");
         }
         break;

      case 6:   // Exit
         printf("Exiting program.\n");
         return 0;

      default:
         printf("Invalid choice!\n");
      }
   }
}
```

2. Write a program to demonstrate Bubble sort techniques using arrays: Arrange following roll list of students in ascending order 8, 73, 15, 86, 54, 22, 34, 3, 18, 51

```
#include <stdio.h>

int main()
{
   int arr[] = {8, 73, 15, 86, 54, 22, 34, 3, 18, 51};
int n = 10;
   int i, j, temp;

   printf("Original Roll Numbers:\n");
for (i = 0; i < n; i++)
      printf("%d ", arr[i]);

   // Bubble Sort     for (i
= 0; i < n - 1; i++)
   {
      for (j = 0; j < n - i - 1; j++)
      {
         if (arr[j] > arr[j + 1])
```

```
        {
            temp = arr[j];
arr[j] = arr[j + 1];
arr[j + 1] = temp; }
      }
   }

   printf("\n\nSorted Roll Numbers (Ascending Order):\n");
for (i = 0; i < n; i++)
      printf("%d ", arr[i]);

   return 0;
}
```

3. Write a program to demonstrate Insertion sort techniques using arrays: Arrange following roll list of students in ascending order 88, 76, 15, 80, 44, 5,37,1, 28, 65

```
#include <stdio.h>

int main()
{
   int arr[] = {88, 76, 15, 80, 44, 5, 37, 1, 28, 65};
int n = 10;
   int i, key, j;

   printf("Original Roll Numbers:\n");
for (i = 0; i < n; i++)
      printf("%d ", arr[i]);

   // Insertion Sort
   for (i = 1; i < n; i++)
   {
      key = arr[i];
j = i - 1;

      while (j >= 0 && arr[j] > key)
      {
         arr[j + 1] = arr[j];
j--;
      }
      arr[j + 1] = key;
   }
```

```
   printf("\n\nSorted Roll Numbers (Ascending Order):\n");
for (i = 0; i < n; i++)
      printf("%d ", arr[i]);

   return 0;   }
```

4. Write a program to demonstrate a singly linked list: Students can be granted membership of the club on request. Store the student roll no. Write functions to a) Add and delete the members.

b) Compute total number of members of club

c) Display members

```
 #include <stdio.h>
#include <stdlib.h>

// Structure definition struct
node
{    int
roll;
   struct node *next;
};

// Head pointer struct
node *head = NULL;

// Function to add member
void addMember(int roll)
{
   struct node *newNode;
   newNode = (struct node *)malloc(sizeof(struct node));

   newNode->roll = roll;
   newNode->next = NULL;

   if (head == NULL)
   {
      head = newNode;
   }
else
   {
      struct node *temp = head;
while (temp->next != NULL)
temp = temp->next;
```

```
        temp->next = newNode;
    }
    printf("Member added successfully.\n");
}

// Function to delete member
void deleteMember(int roll)
{
    struct node *temp = head, *prev = NULL;

    if (head == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    if (head->roll == roll)
    {
        head = head->next;
free(temp);
        printf("Member deleted successfully.\n");
return;
    }

    while (temp != NULL && temp->roll != roll)
    {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL)
    {
        printf("Member not found.\n");
    }
else
    {
        prev->next = temp->next;
free(temp);
        printf("Member deleted successfully.\n");
    }
}

// Function to count members
int countMembers()
```

```
{    int count =
0;
   struct node *temp = head;

   while (temp != NULL)
   {
      count++;        temp
= temp->next;
   }
   return count;
}

// Function to display members
void displayMembers()
{
   struct node *temp = head;

   if (head == NULL)
   {
      printf("No members in the club.\n");
return;
   }

   printf("Club Members Roll Numbers:\n");
while (temp != NULL)
   {
      printf("%d -> ", temp->roll);
      temp = temp->next;
   }
   printf("NULL\n");
}

// Main function int
main()
{
   int choice, roll;

   while (1)
   {
      printf("\n--- CLUB MEMBERSHIP MENU ---\n");
printf("1. Add Member\n");        printf("2. Delete
Member\n");        printf("3. Total Members\n");
printf("4. Display Members\n");        printf("5.
Exit\n");        printf("Enter your choice: ");
```

```
    scanf("%d", &choice);

    switch (choice)
    {
    case 1:
      printf("Enter roll number: ");
scanf("%d", &roll);
      addMember(roll);
break;

    case 2:
      printf("Enter roll number to delete: ");
scanf("%d", &roll);      deleteMember(roll);      break;

    case 3:
      printf("Total members in club: %d\n", countMembers());
break;

    case 4:
      displayMembers();
break;

    case 5:
      printf("Exiting program.\n");
exit(0);

    default:
      printf("Invalid choice!\n");
    }
  }
}
```

5. Write a program to demonstrate a double linked list: Students can be granted membership of the club on request. Store the student roll no. Write functions to

a) Add and delete the members.

b) Compute total number of members of club

c) Display members

d) Display list in reverse order

```
#include <stdio.h>
#include <stdlib.h>
```

```
// Structure of doubly linked list
struct node
{    int
roll;
   struct node *prev;
struct node *next;
};

struct node *head = NULL;

// Add member at end
void addMember(int roll)
{
   struct node *newNode, *temp;
   newNode = (struct node *)malloc(sizeof(struct node));

   newNode->roll = roll;    newNode-
>prev = NULL;    newNode->next =
NULL;

   if (head == NULL)
   {
      head = newNode;
   }
else
   {
      temp = head;       while
(temp->next != NULL)
         temp = temp->next;

      temp->next = newNode;       newNode-
>prev = temp;
   }
   printf("Member added successfully.\n");
}

// Delete member
void deleteMember(int roll)
{
   struct node *temp = head;

   if (head == NULL)
   {
      printf("List is empty.\n");
```

```
        return;
    }

    while (temp != NULL && temp->roll != roll)
temp = temp->next;

    if (temp == NULL)
    {
        printf("Member not found.\n");
        return;
    }

    if (temp->prev != NULL)
        temp->prev->next = temp->next;
    else        head =
temp->next;

    if (temp->next != NULL)
        temp->next->prev = temp->prev;

    free(temp);
    printf("Member deleted successfully.\n");
}

// Count members
int countMembers()
{    int count =
0;
    struct node *temp = head;

    while (temp != NULL)
    {
        count++;        temp
= temp->next;
    }
    return count;
}

// Display members
void displayMembers()
{
    struct node *temp = head;

    if (head == NULL)
```

```
   {
      printf("No members in club.\n");
return;
   }

   printf("Club Members:\n");
   while (temp != NULL)
   {
      printf("%d <-> ", temp->roll);
      temp = temp->next;
   }
   printf("NULL\n");
}

// Display in reverse order
void displayReverse()
{
   struct node *temp = head;

   if (head == NULL)
   {
      printf("No members in club.\n");
return;
   }

   while (temp->next != NULL)
      temp = temp->next;

   printf("Members in Reverse Order:\n");
while (temp != NULL)
   {
      printf("%d <-> ", temp->roll);
      temp = temp->prev;
   }
   printf("NULL\n");
}

// Main function
int main()
{    int choice,
roll;

   while (1)
   {
```

```
    printf("\n--- CLUB MENU ---\n");
printf("1. Add Member\n");
printf("2. Delete Member\n");
printf("3. Count Members\n");
printf("4. Display Members\n");
printf("5. Display Reverse\n");
printf("6. Exit\n");        printf("Enter
choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
    case 1:
      printf("Enter roll number: ");
scanf("%d", &roll);
      addMember(roll);
break;

    case 2:
      printf("Enter roll number to delete: ");
scanf("%d", &roll);
      deleteMember(roll);
break;

    case 3:
      printf("Total members: %d\n", countMembers());
break;

    case 4:
      displayMembers();
break;

    case 5:
displayReverse();
      break;

    case 6:
exit(0);

    default:
      printf("Invalid choice!\n");
    }
  }
}
```

6. Write a program using binary search to find 7.88, 9.22, 7.14 from the following list of CGPA

8.12, 5.67, 9.22, 6.55, 7.88, 5.03, 9.93, 6.79, 8.50, 7.14

```
#include <stdio.h>

// Function to sort array (Bubble Sort)
void sort(float a[], int n)
{
   int i, j;
   float temp;

   for (i = 0; i < n - 1; i++)
   {
      for (j = 0; j < n - i - 1; j++)
      {
         if (a[j] > a[j + 1])
         {
            temp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = temp;
         }
      }
   }
}

// Binary Search function
int binarySearch(float a[], int n, float key)
{
   int low = 0, high = n - 1, mid;

   while (low <= high)
   {
      mid = (low + high) / 2;

      if (a[mid] == key)
         return mid;
      else if (a[mid] < key)
         low = mid + 1;
      else
         high = mid - 1;
   }
   return -1;
}

int main()
```

```
{
   float cgpa[] = {8.12, 5.67, 9.22, 6.55, 7.88, 5.03, 9.93, 6.79, 8.50, 7.14};
   float searchValues[] = {7.88, 9.22, 7.14};
   int n = 10, i, pos;

   // Sort the CGPA list
   sort(cgpa, n);

   printf("Sorted CGPA List:\n");
   for (i = 0; i < n; i++)
      printf("%.2f ", cgpa[i]);

   printf("\n\nBinary Search Results:\n");

   // Search each CGPA
   for (i = 0; i < 3; i++)
   {
      pos = binarySearch(cgpa, n, searchValues[i]);

      if (pos != -1)
         printf("CGPA %.2f found at position %d\n", searchValues[i], pos + 1);
      else
         printf("CGPA %.2f not found\n", searchValues[i]);
   }

   return 0;
}
```

7. Write a program to store following numbers with table size 10 using Hashing

129, 115, 161, 136, 212, 553, 914, 307, 608

```
#include <stdio.h>

#define SIZE 10

int main()
{
   int hashTable[SIZE];
   int keys[] = {129, 115, 161, 136, 212, 553, 914, 307, 608};
int n = 9;
   int i, index;

   // Initialize hash table with -1
for (i = 0; i < SIZE; i++)
hashTable[i] = -1;
```

```
   // Insert keys into hash table
   for (i = 0; i < n; i++)
   {
      index = keys[i] % SIZE;

      // Linear probing for collision
while (hashTable[index] != -1)
      {
         index = (index + 1) % SIZE;
      }
      hashTable[index] = keys[i];
   }

   // Display hash table
   printf("Hash Table (Index : Value)\n");
   for (i = 0; i < SIZE; i++)
   {
      if (hashTable[i] != -1)
         printf("%d : %d\n", i, hashTable[i]);
else
         printf("%d : ---\n", i);
   }

   return 0;
}
```

8. Write a program to perform the following Stack (of size=10) operations: push(): 20, 15, 61, 36, pop(), peek(), pop(), push(): 12, 53, 14, 30, pop(), pop(), peek(), isEmpty(), isFull().

```
#include <stdio.h>

#define SIZE 10

int stack[SIZE];
int top = -1;

// Push operation
void push(int value)
{
   if (top == SIZE - 1)
   {
      printf("Stack Overflow! Cannot push %d\n", value);
```

```
   }
else
   {
      stack[++top] = value;
      printf("Pushed %d\n", value);
   }
}

// Pop operation
void pop()
{    if (top == -
1)
   {
      printf("Stack Underflow! Cannot pop\n");
   }
else
   {
      printf("Popped %d\n", stack[top--]);
   }
}

// Peek operation
void peek()
{    if (top == -
1)
      printf("Stack is empty. No top element\n");
else
      printf("Top element is %d\n", stack[top]);
}

// isEmpty operation
void isEmpty()
{    if (top == -1)
printf("Stack is Empty\n");
else
      printf("Stack is NOT Empty\n");
}

// isFull operation
void isFull()
{    if (top == SIZE - 1)
printf("Stack is Full\n");
else
      printf("Stack is NOT Full\n");
```

```
}

int main()
{
   // Given operations
   push(20);
push(15);    push(61);
push(36);

   pop();
   peek();

   pop();

   push(12);
push(53);    push(14);
push(30);

   pop();
pop();

   peek();
isEmpty();
   isFull();

   return 0;
}
```

9. Write a program to convert Infix expression A + (B * C) / D to Postfix expression.

```
#include <stdio.h>
#include <ctype.h>

char stack[20];
int top = -1;

// Push operator to stack
void push(char ch)
{
   stack[++top] = ch;
}

// Pop operator from stack char
pop()
{
```

```
   return stack[top--];
}

// Check operator precedence
int precedence(char ch)
{
   if (ch == '+' || ch == '-')
return 1;    if (ch == '*' ||
ch == '/')       return 2;
   return 0;
}

int main()
{
   char infix[] = "A+(B*C)/D";
char postfix[20];    int i = 0,
k = 0;
   char ch;

   while ((ch = infix[i++]) != '\0')
   {
      // If operand, add to postfix
      if (isalnum(ch))
      {
         postfix[k++] = ch;
      }
      // If '(' push to stack
      else if (ch == '(')
      {
         push(ch);
      }
      // If ')', pop until '('
      else if (ch == ')')
      {
         while (stack[top] != '(')
postfix[k++] = pop();
         pop(); // Remove '('
      }
      // If operator
      else
      {
         while (top != -1 && precedence(stack[top]) >= precedence(ch))
postfix[k++] = pop();           push(ch);
      }
```

```
    }

    // Pop remaining operators
while (top != -1)
        postfix[k++] = pop();

    postfix[k] = '\0';

    printf("Postfix Expression: %s\n", postfix);

    return 0;
}
```

10. Write a program to check the well form-ness of ( ), [ ], { } parenthesis :- whether a given expression is balanced or unbalanced.

```
#include <stdio.h>

#define SIZE 100

char stack[SIZE];
int top = -1;

// Push operation
void push(char ch)
{
    stack[++top] = ch;
}

// Pop operation
char pop()
{
    return stack[top--];
}

// Function to check matching pair
int isMatching(char open, char close)
{
    if (open == '(' && close == ')') return 1;
if (open == '[' && close == ']') return 1;
if (open == '{' && close == '}') return 1;
return 0;
```

```
}

// Check balanced expression
int isBalanced(char exp[])
{    int i =
0;
   char ch;

   while ((ch = exp[i++]) != '\0')
   {
      // If opening bracket, push
      if (ch == '(' || ch == '[' || ch == '{')
      {
         push(ch);
      }
      // If closing bracket
      else if (ch == ')' || ch == ']' || ch == '}')
      {
         if (top == -1)   // No opening bracket
return 0;

         if (!isMatching(pop(), ch))
            return 0;
      }
   }
   return (top == -1);
}

int main()
{
   char exp[100];

   printf("Enter an expression: ");
scanf("%s", exp);

   if (isBalanced(exp))
      printf("Expression is BALANCED\n");
else
      printf("Expression is UNBALANCED\n");

   return 0; }
```

11. Write a program to simulate the Queues for jobs of an operating system that does not use priorities, write functions to add jobs and delete jobs from the queue.

```
#include <stdio.h>

#define SIZE 10

int queue[SIZE];
int front = -1, rear = -1;

void addJob(int job)
{
   if (rear == SIZE - 1)
   {
      printf("Queue Overflow! Cannot add job %d\n", job);
   }
   else
   {
      if (front == -1)
         front = 0;

      queue[++rear] = job;
      printf("Job %d added to queue\n", job);
   }
}
void deleteJob()
{
   if (front == -1 || front > rear)
   {
      printf("Queue Underflow! No job to delete\n");
   }
   else
   {
      printf("Job %d deleted from queue\n", queue[front++]);
   }
}

void displayQueue()
{    int
i;

   if (front == -1 || front > rear)
   {
      printf("Queue is empty\n");
return;
   }
```

```
   printf("Jobs in Queue:\n");
for (i = front; i <= rear; i++)
printf("%d ", queue[i]);
printf("\n");
}

int main()
{
   int choice, job;

   while (1)
   {
      printf("\n--- OS JOB QUEUE MENU ---\n");
      printf("1. Add Job\n");
printf("2. Delete Job\n");
printf("3. Display Jobs\n");
printf("4. Exit\n");        printf("Enter
your choice: ");
      scanf("%d", &choice);

      switch (choice)
      {
      case 1:
printf("Enter Job ID: ");
scanf("%d", &job);
addJob(job);            break;

      case 2:
deleteJob();
         break;

      case 3:
displayQueue();
         break;

      case 4:
         printf("Exiting program.\n");
         return 0;

      default:
printf("Invalid choice!\n");
      }
   }
}
```

12. Write a program to simulate double-ended queue (deque) with following functions to add and delete elements from the deque [10, 20, 30, 40, 50].

addFront(5), addRear(60), deleteFront(), deleteRear(), addFront(7), addRear(55), deleteFront(), deleteRear()

```
#include <stdio.h>


#define SIZE 20

int deque[SIZE];
int front = 0, rear = 4;

void addFront(int x)
{
   front--;
deque[front] = x;
   printf("addFront(%d)\n", x);
}

void addRear(int x)
{
   rear++;
deque[rear] = x;
   printf("addRear(%d)\n", x);
}

void deleteFront()
{
   printf("deleteFront() -> %d\n", deque[front]);
   front++;
}

void deleteRear()
{
   printf("deleteRear() -> %d\n", deque[rear]);
   rear--;
}

void display()
{
   int i;
```

```
   printf("Deque: ");     for (i
= front; i <= rear; i++)
printf("%d ", deque[i]);
   printf("\n");
}

int main()
{
   deque[0] = 10;
deque[1] = 20;     deque[2]
= 30;     deque[3] = 40;
deque[4] = 50;

   printf("Initial ");
   display();

   addFront(5);
   display();

   addRear(60);
   display();

   deleteFront();
display();

   deleteRear();
   display();

   addFront(7);
   display();

   addRear(55);
   display();

   deleteFront();
   display();

   deleteRear();
   display();

   return 0;
}
```

13. Construct the Binary tree using linked list and perform the inorder (20, 30, 40, 50, 60, 70, 80), preorder, post order traversals

```
#include <stdio.h>
#include <stdlib.h>

struct node
{    int data;
struct node *left;
   struct node *right;
};

struct node* createNode(int value)
{
   struct node *newNode;
   newNode = (struct node*)malloc(sizeof(struct node));

   newNode->data = value;    newNode-
>left = NULL;
   newNode->right = NULL;

   return newNode;
}

void inorder(struct node *root)
{
   if (root != NULL)
   {
      inorder(root->left);
printf("%d ", root->data);
      inorder(root->right);
   }
}

void preorder(struct node *root)
{
   if (root != NULL)
   {
      printf("%d ", root->data);       preorder(root-
>left);
      preorder(root->right);
   }
}
```

```
void postorder(struct node *root)
{
   if (root != NULL)
   {
      postorder(root->left);
postorder(root->right);        printf("%d
", root->data);
   }
}

int main()
{
   struct node *root = createNode(50);

   root->left = createNode(30);
   root->right = createNode(70);

   root->left->left = createNode(20);
   root->left->right = createNode(40);

   root->right->left = createNode(60);
   root->right->right = createNode(80);

   printf("Inorder Traversal: ");
inorder(root);

   printf("\nPreorder Traversal: ");
preorder(root);

   printf("\nPostorder Traversal: ");
postorder(root);

   return 0;
}
```

14. Write a program to demonstrate a tree for a book that consists of chapters, the chapters consist of sections and sections consist of subsections. Construct a tree and print the nodes.

```
#include <stdio.h>
#include <stdlib.h>

// Tree node structure (First Child - Next Sibling) struct
node
{
   char name[50];
struct node *child;
   struct node *sibling;
};

struct node* createNode(char name[])
{
   struct node *newNode;
   newNode = (struct node*)malloc(sizeof(struct node));

   strcpy(newNode->name, name);
newNode->child = NULL;     newNode->sibling
= NULL;     return newNode;
}

void printTree(struct node *root, int level)
{    int
i;

   if (root == NULL)
      return;

   for (i = 0; i < level; i++)
      printf("  ");

   printf("%s\n", root->name);

   printTree(root->child, level + 1);
   printTree(root->sibling, level);
}

int main()
{
   struct node *book = createNode("Book");

   struct node *ch1 = createNode("Chapter 1");
struct node *ch2 = createNode("Chapter 2");
```

```
   book->child = ch1;
   ch1->sibling = ch2;


   struct node *sec11 = createNode("Section 1.1");
struct node *sec12 = createNode("Section 1.2");

   ch1->child = sec11;     sec11->sibling = sec12;     struct
node *sub111 = createNode("Subsection 1.1.1");     struct
node *sub112 = createNode("Subsection 1.1.2");

   sec11->child = sub111;
   sub111->sibling = sub112;

   struct node *sub121 = createNode("Subsection 1.2.1");
sec12->child = sub121;

   struct node *sec21 = createNode("Section 2.1");     ch2-
>child = sec21;


   struct node *sub211 = createNode("Subsection 2.1.1");
sec21->child = sub211;

   printf("Book Structure:\n");
printTree(book, 0);

   return 0;  }
```

15. Write a program to construct a Binary search tree by inserting the values in the order given. Begin with an empty binary search tree and after constructing the tree a) Insert new node

b) Delete a node.

c) Search a value

```
#include <stdio.h> #include
<stdlib.h>
struct node
```

```
{    int data;
struct node *left;
   struct node *right;
};
struct node* createNode(int value)
{
   struct node *newNode;
   newNode = (struct node*)malloc(sizeof(struct node));

   newNode->data = value;    newNode-
>left = NULL;
   newNode->right = NULL;

   return newNode;
}

struct node* insert(struct node *root, int value)
{
   if (root == NULL)        return
createNode(value);    if (value
< root->data)        root->left =
insert(root->left, value);    else
if (value > root->data)      root-
>right    =    insert(root->right,
value);

   return root;
}

struct node* findMin(struct node *root)
{
   while (root->left != NULL)
root = root->left;
   return root;
}

struct node* deleteNode(struct node *root, int value)
{
   if (root == NULL)
      return root;

   if (value < root->data)
      root->left = deleteNode(root->left, value);
else if (value > root->data)       root->right =
deleteNode(root->right, value);    else
```

```
   {
      if (root->left == NULL)
      {
         struct node *temp = root->right;
         free(root);
         return temp;
      }
      else if (root->right == NULL)
      {
         struct node *temp = root->left;
         free(root);
         return temp;
      }

      struct node *temp = findMin(root->right);        root-
>data = temp->data;
      root->right = deleteNode(root->right, temp->data);
   }
   return root;
}
int search(struct node *root, int key)
{
   if (root == NULL)        return 0;
if (root->data == key)        return
1;     else if (key < root->data)
return search(root->left, key);
else
      return search(root->right, key);
}

// Inorder traversal (to display tree)
void inorder(struct node *root)
{
   if (root != NULL)
   {
      inorder(root->left);
printf("%d ", root->data);
      inorder(root->right);
   }
}

int main()
{
   struct node *root = NULL;
   int choice, value;
```

```
    while (1)
    {
        printf("\n--- BST MENU ---\n");
printf("1. Insert\n");        printf("2.
Delete\n");        printf("3.
Search\n");        printf("4. Display
(Inorder)\n");        printf("5.
Exit\n");        printf("Enter your
choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
        case 1:
            printf("Enter value to insert: ");
scanf("%d", &value);            root =
insert(root, value);            break;
        case 2:
            printf("Enter value to delete: ");
scanf("%d", &value);            root =
deleteNode(root, value);            break;

        case 3:
            printf("Enter value to search: ");
scanf("%d", &value);            if
(search(root, value))
printf("Value found in BST\n");
else
                printf("Value not found in BST\n");
break;

        case 4:
            printf("Inorder Traversal: ");
inorder(root);            printf("\n");
            break;

        case 5:
exit(0);

        default:
printf("Invalid choice!\n");
        }
    }
}
```

16. Write a program to perform the Depth First Search using adjacency list or adjacency matrix, use the map of the historical places around Pune city as the graph, and identify the famous landmarks as nodes. (Shaniwar Wada, Aga Khan Palace, Pataleshwar Cave Temple, Lal Mahal, Shinde Chhatri, Sinhagad Fort, Parvati Hill Temple, Vishrambaug Wada, Bhide Wada)

```
#include <stdio.h>

#define N 9

int visited[N] = {0};

char places[N][40] = {
   "Shaniwar Wada",
   "Aga Khan Palace",
   "Pataleshwar Cave Temple",
   "Lal Mahal",
   "Shinde Chhatri",
   "Sinhagad Fort",
   "Parvati Hill Temple",
   "Vishrambaug Wada",
   "Bhide Wada"
};

void DFS(int graph[N][N], int start)
{
   int i;
   printf("%s\n", places[start]);
   visited[start] = 1;

   for (i = 0; i < N; i++)
   {
      if (graph[start][i] == 1 && visited[i] == 0)
      {
         DFS(graph, i);
      }
   }
}

int main()
```

```
{
   int graph[N][N] = {
      // SW  AK  PC  LM  SC  SF  PH  VW  BW
      { 0, 0, 1, 1, 0, 0, 1, 1, 1 },
      { 0, 0, 0, 0, 1, 0, 0, 0, 0 },
      { 1, 0, 0, 0, 0, 0, 0, 0, 0 },
      { 1, 0, 0, 0, 0, 0, 0, 0, 1 },
      { 0, 1, 0, 0, 0, 0, 0, 0, 0 },
      { 0, 0, 0, 0, 0, 0, 1, 0, 0 },
      { 1, 0, 0, 0, 0, 1, 0, 0, 0 },
      { 1, 0, 0, 0, 0, 0, 0, 0, 0 },
      { 1, 0, 0, 1, 0, 0, 0, 0, 0 }
   };

   printf("Depth First Search Traversal starting from Shaniwar Wada:\n\n");
   DFS(graph, 0);

   return 0;
}
```

17. Write a program to perform the Breadth First Search using adjacency list or adjacency matrix, use the map of the historical places around Pune city as the graph, and identify the famous landmarks as nodes. (Shaniwar Wada, Aga Khan Palace, Pataleshwar Cave Temple, Lal Mahal, Shinde Chhatri, Sinhagad Fort, Parvati Hill Temple, Vishrambaug Wada, Bhide Wada)

```
#include <stdio.h>

#define N 9
#define MAX 100

int queue[MAX], front = -1, rear = -1;

void enqueue(int x)
{
   queue[++rear] = x;
}

int dequeue()
{
   return queue[++front];
}
```

```
int isEmpty()
{
   return front == rear;
}

char places[N][40] = {
"Shaniwar Wada",
   "Aga Khan Palace",
   "Pataleshwar Cave Temple",
   "Lal Mahal",
   "Shinde Chhatri",
   "Sinhagad Fort",
   "Parvati Hill Temple",
   "Vishrambaug Wada",
   "Bhide Wada"
};
void BFS(int graph[N][N], int start)
{
   int visited[N] = {0};
int i;     enqueue(start);
   visited[start] = 1;

   while (!isEmpty())
   {
      int node = dequeue();
      printf("%s\n", places[node]);

      for (i = 0; i < N; i++)
      {
         if (graph[node][i] == 1 && !visited[i])
         {
            enqueue(i);
            visited[i] = 1;
         }
      }
   }
}

int main()
{
   // Adjacency matrix representation
int graph[N][N] = {
      // SW  AK  PC  LM  SC  SF  PH  VW  BW
```

```
        { 0, 0, 1, 1, 0, 0, 1, 1, 1 },
        { 0, 0, 0, 0, 1, 0, 0, 0, 0 },
        { 1, 0, 0, 0, 0, 0, 0, 0, 0 },
        { 1, 0, 0, 0, 0, 0, 0, 0, 1 },
        { 0, 1, 0, 0, 0, 0, 0, 0, 0 },
        { 0, 0, 0, 0, 0, 0, 1, 0, 0 },
        { 1, 0, 0, 0, 0, 1, 0, 0, 0 },
        { 1, 0, 0, 0, 0, 0, 0, 0, 0 },
        { 1, 0, 0, 1, 0, 0, 0, 0, 0 }
    };

    printf("Breadth First Search Traversal starting from Shaniwar Wada:\n\n");
BFS(graph, 0);

    return 0;
}
```